Challenge:	Sorting Hat Blocks
Category:	Web Exploitation / Cryptography / AES-ECB
Difficulty:	Medium
Tools Used:	Browser, Terminal (Kali Linux / Git Bash / Python)

Summary

This challenge is about breaking a weak encryption system.The server takes whatever message you type, secretly adds a hidden flag to the end of it and then encrypts the whole thing. Your job is to figure out what that hidden flag is without ever being told the encryption key. The encryption mode being used is called AES-ECB which has a well known flaw that makes this possible. By sending carefully chosen inputs and observing the outputs, you can recover the hidden flag one character at a time.

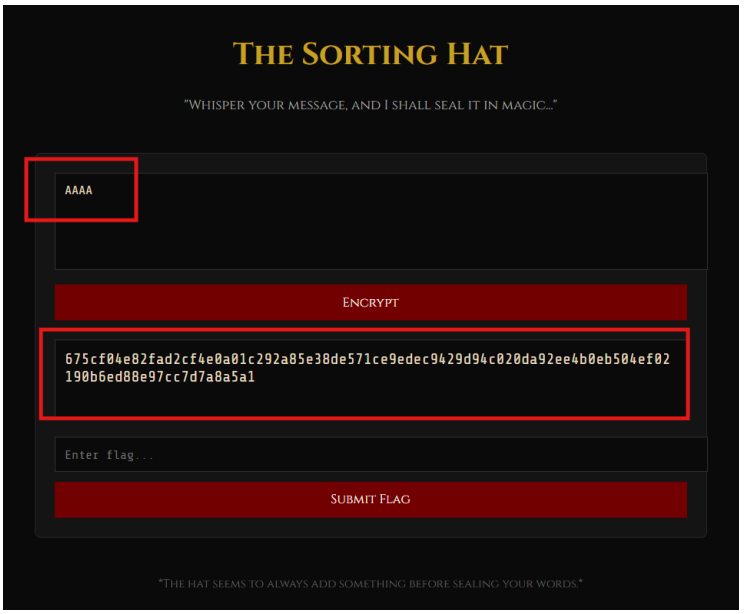
Problem Statement

Ah… another curious mind.
The Sorting Hat has long guarded ancient magic, revealing truths only to those patient enough to observe its patterns.
It accepts any message you offer…
but the output it returns is never entirely your own.
Something is always appended.
Something hidden within the spell.
The transformation is consistent. Predictable, even if you watch closely enough.
Many have tried to extract what the Hat conceals.
None have succeeded.
Can you uncover the spell hidden within its magic?

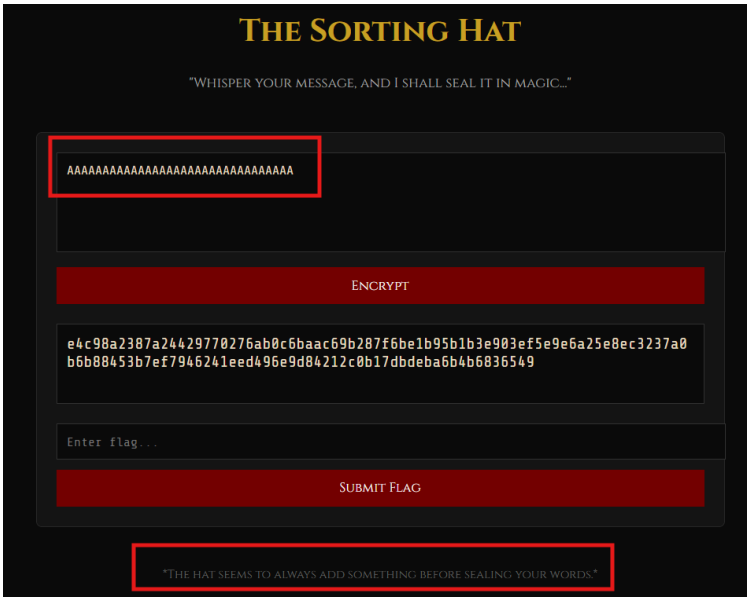
Steps to Solve

1. After opening the webpage you will see a simple input field, type anything and hit submit. The server responds with something like: 3a4f1bc2e9d07f56a1c3…
2. This long string is written in hexadecimal which is a way of representing raw data using only characters 0–9 and a–f.
3. Every two hex characters represent one byte of data. So if you see 32 hex characters that's 16 bytes which is exactly one AES block.
4. You need to confirm the block size experimentally. The server encrypts (your_input) + (hidden_secret) and the total must always be a multiple of 16 bytes,so padding is added to fill the last block.
5. As you keep adding characters, at some point the data overflows into a new block and the ciphertext suddenly gets longer.
6. Send these inputs one at a time and note the ciphertext length each time:
   A
   AA
   AAA
   AAAA... keep adding one A at a time
   We use A simply because it's easy to count and repeat otherwise any character works.

You're watching for when the output suddenly becomes longer. The amount it jumps by (in hex characters ÷ 2) is the block size.



7. The ciphertext jumps by 32 hex characters → block size = 16 bytes, confirming standard AES.
8. You might expect: send 32 identical characters and if the first two ciphertext blocks match its ECB. But when you try it:
   AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
   The first two blocks are not identical. Does this mean it's not ECB?
   Not quite. There's a hint on the page: "The hat seems to always add something before sealing your words." This reveals that the server is also adding an unknown prefix before your input, not just appending at the end. So what's actually being encrypted is: Encrypt(unknown_prefix + your_input + hidden_flag)



9. The prefix shifts your A's across block boundaries so they don't align cleanly. The fix: keep adding more A's until you force two full identical blocks somewhere in the middle of the ciphertext regardless of where the prefix ends.

10. AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA... keep adding A's …

The moment you see two consecutive identical 32-character blocks in the output, your A's have overflowed past the prefix and two clean blocks are aligning perfectly.
Observation: Two identical blocks appear → ECB confirmed.

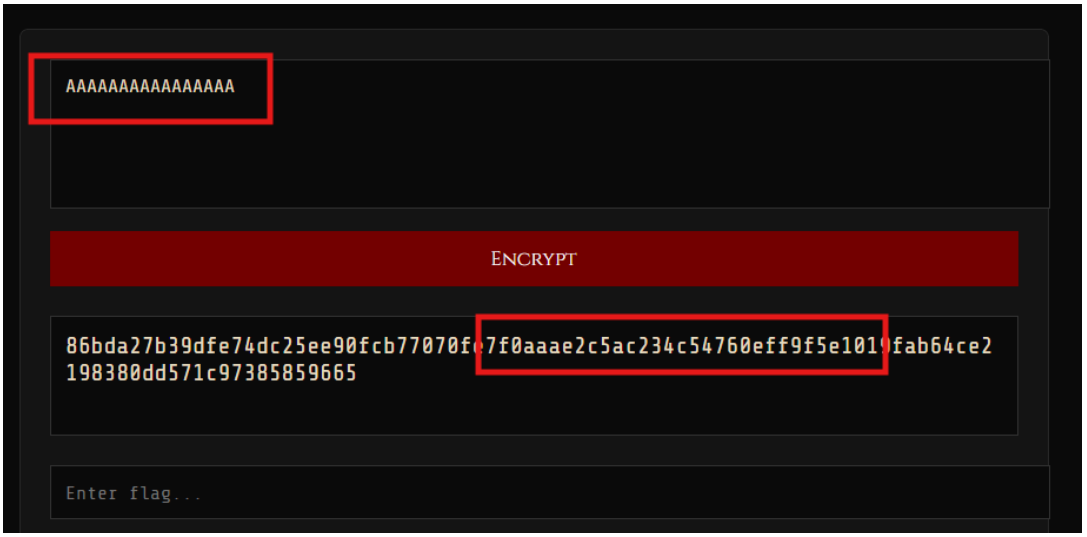
11. Hint 1 connection: "Try repeating your input... does the magic behave the same way?" - the matching blocks are ECB revealing itself.
12. Hint 2 connection: "Some ciphers reveal patterns when identical blocks are used." - this repeated block is that pattern.
13. Two identical blocks appeared when sending 33 A's, meaning 32 bytes of aligned input were achieved after compensating for a 1 byte unknown prefix. This allows us to correctly align our input for the byte-at-a-time ECB attack.
14. AES-ECB always produces the same ciphertext for the same 16 byte input.
    So if we can control 15 bytes of a block and leave 1 byte unknown, we can brute force that last byte.
15. From the previous step, we found: prefix length ≈ 1 byte
16. To align properly, we need to add padding so our input starts at a clean block boundary.
17. So we send: **"A" * 15**
18. Why 15?
    => 1 byte prefix + 15 A's = 16 bytes → fills block 1 exactly
19. Now our controlled input starts cleanly from block 2
20. Now block 2 looks like:
    AAAAAAAAAAAAAAA?
    15 A's (known)
    ? = first unknown byte of the flag

21. If we can recreate this exact 16-byte input, ECB will give the same ciphertext block.
22. Recovering the First Character :
    To extract the first byte of the hidden flag, we use a byte-at-a-time ECB attack by carefully controlling the input and comparing ciphertext blocks.

23. We begin by sending 15 A's:
    AAAAAAAAAAAAAAA
    Because we already aligned our input in the previous step, the encryption now looks like:
    [prefix][AAAAAAAAAAAAAAA][flag_char_1][...]

24. This means the second block of the ciphertext is:
    AAAAAAAAAAAAAAA[flag_char_1]

25. We extract and save this block - this becomes our target block.



26. Next, we attempt to recreate this exact block by guessing the last character.
27. We send inputs of the form:
    AAAAAAAAAAAAAAA + guess

28. Trying all possible characters:
    AAAAAAAAAAAAAAA A
    AAAAAAAAAAAAAAA B
    AAAAAAAAAAAAAAA C…

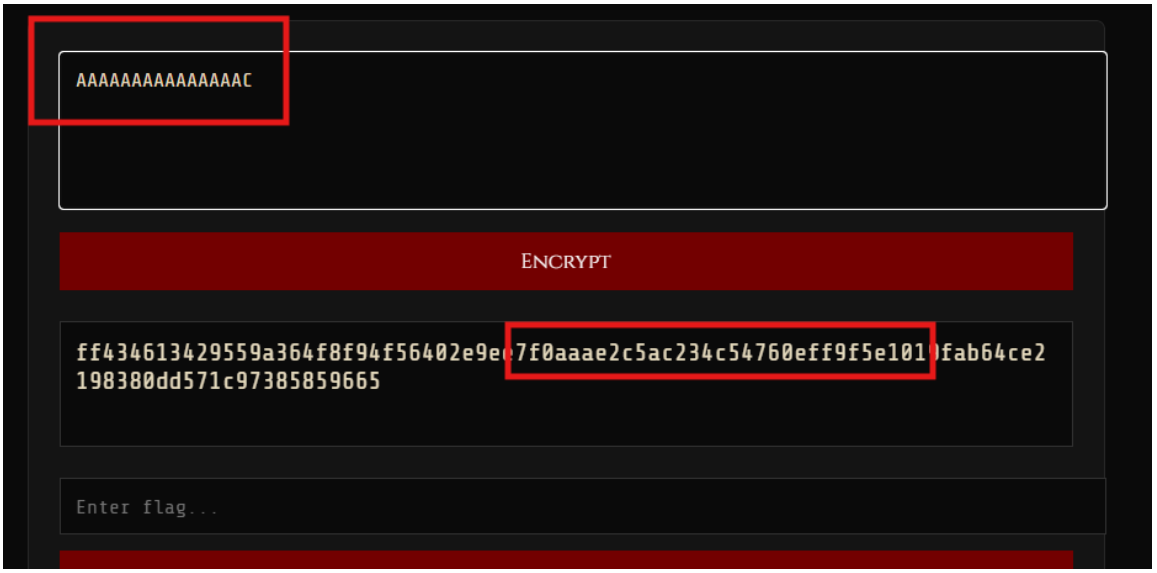
29. For each request:
    Extract Block 2 from the ciphertext

30. Compare it with the saved target block

31. Now when the ciphertext block matches the target block, it means the input we constructed is identical to the original.

32. For example:
    AAAAAAAAAAAAAAC
    If this produces the same ciphertext block:

    AAAAAAAAAAAAAAA[flag_char_1]



The block matches which confirms the first character of the flag is C

33. After successfully recovering the first character, the same logic is repeated for the remaining bytes.
34. Instead of guessing just one unknown byte, we now include the already recovered characters in our input.
35. For the second character, the block becomes:

    AAAAAAAAAAAAAA[C][?]
    14 A's
    1 known character (C)
    1 unknown byte

36. We again brute-force the last position by trying all possible characters and matching the ciphertext block.

Round	Input Structure
1	AAAAAAAAAAAAAA?
2	AAAAAAAAAAAAA C ?
3	AAAAAAAAAAAA Cr ?
...	...
16	full block recovered

37. Each time:
    Reduce padding by 1 A
    Add known characters
    Brute-force last byte

38. After 16 rounds, you've recovered all 16 characters of the first block of the flag. For the second block, you shift to comparing block 3 of the ciphertext and use your 16 recovered characters as the known bytes going forward.

    Hint 4 connection: "Shift it carefully — you can reveal the hidden secret one byte at a time." — "shift" is literally reducing padding by one each round, pushing the unknown byte into the last position.

39. Now that you fully understand the manual logic, it should be clear that doing this for every character of the flag which could be 30+ characters means hundreds of requests. This is where a Python script becomes essential.

    The script does exactly what you just did manually but in a loop:
    Calculates the correct padding for each round (accounting for prefix)
    Sends the target request and saves the correct block
    Loops through all 256 possible byte values as guesses
    Sends each guess and compares the block
    Records the matched character, appends it to the known flag and moves to the next position

    The script finishes in seconds and prints the full flag, the same logic you walked through above, just automated.

40. The required script with comments:
    import requests
    from concurrent.futures import ThreadPoolExecutor, as_completed

```
# ------------------------------------------------------------
# TARGET URL - change this to your challenge server's encrypt endpoint
# For localhost: "http://127.0.0.1:5000/encrypt"
# For remote:    "http://<challenge-ip>:<port>/encrypt"
# ------------------------------------------------------------
URL = "http://127.0.0.1:5000/encrypt"

# Session keeps the connection alive across requests (faster than opening a new connection each time)
SESSION = requests.Session()

def oracle(attacker_input: bytes) -> bytes:
    """
    Sends our chosen input to the server and gets back the ciphertext.
    The server does: Encrypt(prefix + our_input + hidden_flag)
    We send bytes as hex string because the server expects hex-encoded input.
    """
    r = SESSION.post(URL, json={"data": attacker_input.hex()})

    # If server is rate limiting us, wait 1 second and retry
    if r.status_code == 429:
        import time; time.sleep(1)
        return oracle(attacker_input)

    if r.status_code != 200:
        raise RuntimeError(f"{r.status_code}: {r.text}")

    # Server returns hex ciphertext — convert it back to raw bytes
    return bytes.fromhex(r.text)

def detect_block_size() -> int:
    """
    Finds the block size by sending increasing A's until the ciphertext
    suddenly gets longer. The jump in length = the block size.
    Example: if ciphertext jumps by 16 bytes, block size = 16.
    """
```

```
    initial_len = len(oracle(b""))  # baseline ciphertext length with empty input

    for i in range(1, 65):
        if len(oracle(b"A" * i)) > initial_len:
            block_size = len(oracle(b"A" * i)) - initial_len
            print(f"[*] Block size detected: {block_size} bytes")
            return block_size

    raise ValueError("Could not detect block size")

def confirm_ecb(block_size: int) -> bool:
    """
    Confirms ECB mode by sending two identical blocks of A's.
    In ECB mode, identical plaintext blocks always produce identical ciphertext blocks.
    If block 0 == block 1 in the output, it's ECB.
    Note: This uses 2 * block_size A's WITHOUT accounting for prefix,
    so it works only if the prefix is a multiple of block_size.
    The main attack handles prefix alignment properly.
    """
    ct = oracle(b"A" * (block_size * 2))
    blocks = [ct[i:i+block_size] for i in range(0, len(ct), block_size)]
    result = blocks[0] == blocks[1]
    print(f"[*] ECB mode confirmed: {result}")
    return result


def detect_secret_length(block_size: int) -> int:
    """
    Finds the exact length of the hidden flag/secret.
    Strategy: keep adding A's until the ciphertext grows by one block.
    The moment it grows, the padding just pushed into a new block —
    meaning before that push, the last block was exactly full.
    secret_length = base_ciphertext_length - number_of_A's_added
    """
    base_len = len(oracle(b""))  # ciphertext length with no input

    for i in range(1, block_size + 1):
        if len(oracle(b"A" * i)) > base_len:
            secret_len = base_len - i
            print(f"[*] Secret (flag) length: {secret_len} bytes")
            return secret_len

    raise ValueError("Could not detect secret length")


def crack_byte(i, block_size, recovered, prefix, target_block, block_index):
    """
    Recovers one unknown byte of the flag by trying all 256 possible values.

    How it works:
    - We already know (block_size - 1) bytes in the target block
    - The last byte is unknown (next flag character)
    - We try every possible byte value (0-255) as the last byte
    - When our guess produces the same ciphertext block as the real one, we found it

    Uses parallel threads (16 at a time) to speed up the 256 guesses.
```

```
    """
    def try_candidate(candidate):
        # Build input: padding A's + all recovered bytes so far + our guess
        test_input = prefix + recovered + bytes([candidate])
        test_ct = oracle(test_input)
        # Extract the specific block we're targeting
        test_block = test_ct[block_index * block_size:(block_index + 1) * block_size]
        # If our guessed block matches the real target block, we found the byte
        return candidate if test_block == target_block else None

    # Run all 256 guesses in parallel using 16 threads
    with ThreadPoolExecutor(max_workers=16) as ex:
        futures = {ex.submit(try_candidate, c): c for c in range(256)}
        for future in as_completed(futures):
            result = future.result()
            if result is not None:
                return result  # return as soon as we find a match

    return None  # no match found (shouldn't happen in a working ECB oracle)


def byte_at_a_time(block_size: int, secret_len: int) -> bytes:
    """
    Main attack loop — recovers the full flag one byte at a time.

    For each byte position i:
    1. Send (block_size - 1 - i % block_size) A's as padding
       This pushes exactly one unknown flag byte to the last position of the target block
    2. Save the target block from the real ciphertext
    3. Brute-force all 256 values for that last byte
    4. Append the matched byte to our recovered flag
    5. Move to the next byte — padding reduces by 1 each round
    """
    recovered = b""
    print(f"[*] Recovering {secret_len} bytes using byte-at-a-time attack...\n")

    for i in range(secret_len):
        # Which block contains the byte we're currently targeting
        block_index = i // block_size

        # How many A's to pad so the unknown byte sits at the last position
        # of the target block. Reduces by 1 each round (the "shift")
        pad_len = block_size - 1 - (i % block_size)
        prefix = b"A" * pad_len

        # Get the real ciphertext with this padding
        real_ct = oracle(prefix)

        # Extract just the block we're targeting
        target_block = real_ct[block_index * block_size:(block_index + 1) * block_size]

        # Try all 256 values and find which one matches
        candidate = crack_byte(i, block_size, recovered, prefix, target_block, block_index)

        if candidate is not None:
            recovered += bytes([candidate])
```

```
            # Show printable character or hex if non-printable
            printable = chr(candidate) if 32 <= candidate <= 126 else f"\\x{candidate:02x}"
                                        print(f"[+]    Byte    {i+1:>2}/{secret_len}:    '{printable}'       →
        {recovered.decode(errors='replace')}")
        else:
            print(f"[!] Could not recover byte {i+1} — stopping here")
            break

    return recovered


if __name__ == "__main__":
    # Step 1: Find block size by watching ciphertext length grow
    block_size = detect_block_size()

    # Step 2: Confirm it's ECB mode (identical blocks = ECB)
    if not confirm_ecb(block_size):
        print("[!] Server is not using ECB mode — attack won't work")
        exit(1)

    # Step 3: Find how long the hidden flag is
    secret_len = detect_secret_length(block_size)

    # Step 4: Recover the flag byte by byte
    secret = byte_at_a_time(block_size, secret_len)

    print(f"\n[+] FLAG: {secret.decode(errors='replace')}")
```

```
[+] Byte 24/37: 'u'  →  Cruxhunt{Exp3cto_Patr0nu
[+] Byte 25/37: 'm'  →  Cruxhunt{Exp3cto_Patr0num
[+] Byte 26/37: '_'  →  Cruxhunt{Exp3cto_Patr0num_
[+] Byte 27/37: 'E'  →  Cruxhunt{Exp3cto_Patr0num_E
[+] Byte 28/37: 'C'  →  Cruxhunt{Exp3cto_Patr0num_EC
[+] Byte 29/37: 'B'  →  Cruxhunt{Exp3cto_Patr0num_ECB
[+] Byte 30/37: '_'  →  Cruxhunt{Exp3cto_Patr0num_ECB_
[+] Byte 31/37: 'm'  →  Cruxhunt{Exp3cto_Patr0num_ECB_m
[+] Byte 32/37: '4'  →  Cruxhunt{Exp3cto_Patr0num_ECB_m4
[+] Byte 33/37: 's'  →  Cruxhunt{Exp3cto_Patr0num_ECB_m4s
[+] Byte 34/37: 't'  →  Cruxhunt{Exp3cto_Patr0num_ECB_m4st
[+] Byte 35/37: '3'  →  Cruxhunt{Exp3cto_Patr0num_ECB_m4st3
[+] Byte 36/37: 'r'  →  Cruxhunt{Exp3cto_Patr0num_ECB_m4st3r
[+] Byte 37/37: '}'  →  Cruxhunt{Exp3cto_Patr0num_ECB_m4st3r}

[+] FLAG: Cruxhunt{Exp3cto_Patr0num_ECB_m4st3r}
```

The following output shows the automated byte-at-a-time ECB attack in action.
Each byte of the hidden flag is recovered sequentially by matching ciphertext blocks.

As seen above, the script successfully reconstructs the flag character by character until completion.

Final Flag : Cruxhunt{Exp3cto_Patr0num_ECB_m4st3r}